

LAB GUIDELINES

Lab 21 — Multi-Factor Authentication with TOTP (PAM)

Maps to CompTIA Server+ objective **3.3** — Multifactor authentication (something you know / have / are).

You will add a TOTP **second factor** to SSH logins. The password is the *something you know*; the TOTP code from a phone app is the *something you have*.

Run all commands on <https://killercoda.com/playgrounds/scenario/ubuntu>

Required software (free, apt): libpam-google-authenticator, qrencode, oathtool, openssh-server

```
apt update && apt install -y libpam-google-authenticator qrencode oathtool openssh-server
systemctl start ssh
```

Step 1 — Create a non-privileged user

```
useradd -m -s /bin/bash mfauser
echo 'mfauser:MfaPass!1' | chpasswd
```

Step 2 — Provision the TOTP secret for that user

Run as the user so the secret lives in their home dir:

```
sudo -u mfauser google-authenticator -t -d -f -r 3 -R 30 -w 17 -Q UTF8
```

Flags meaning:

- `-t` time-based (TOTP) instead of HOTP
- `-d` disallow code reuse
- `-f` write `.google_authenticator` non-interactively
- `-r 3 -R 30` rate-limit 3 logins / 30 seconds
- `-w 17` allow a 17-window time skew ($\sim \pm 2.5$ min)
- `-Q UTF8` print a QR code to the terminal

A user would scan the QR in **Google Authenticator / Authy / FreeOTP / 1Password**. The file `/home/mfauser/.google_authenticator` holds the shared secret + scratch codes.

```
head -3 /home/mfauser/.google_authenticator
```

Step 3 — Wire PAM to require TOTP for SSH

```
echo 'auth required pam_google_authenticator.so nullok' > /etc/pam.d/sshd.mfa
sed -i '1i @include sshd.mfa' /etc/pam.d/sshd
grep -q '^@include sshd.mfa' /etc/pam.d/sshd || sed -i '1i @include sshd.mfa' /etc/pam.d/sshd

sed -i 's/^#*KbdInteractiveAuthentication.*KbdInteractiveAuthentication yes/' /etc/ssh/sshd_config
sed -i 's/^#*ChallengeResponseAuthentication.*ChallengeResponseAuthentication yes/' /etc/ssh/sshd_config
grep -q 'AuthenticationMethods' /etc/ssh/sshd_config || \
    echo 'AuthenticationMethods publickey,keyboard-interactive password,keyboard-interactive' >> /etc/ssh/sshd_config
systemctl restart ssh
```

Step 4 — Generate the current TOTP from the CLI (for testing)

The shared secret is on the first line of `.google_authenticator`:

```
SECRET=$(sudo head -1 /home/mfauser/.google_authenticator)
oathtool --totp -b "$SECRET"
```

Output is the 6-digit code that's valid for the next ~30 s. In a real workflow this comes from the phone app.

Step 5 — Test the login

```
ssh -o PreferredAuthentications=keyboard-interactive,password -o PubkeyAuthentication=no mfauser@12
# Prompt 1: Password    → MfaPass!1      (something you know)
# Prompt 2: Verification code → from oathtool / phone app (something you have)
```

A wrong code denies access **even with the right password**. That's MFA working.

Step 6 — Scratch codes (account recovery)

The file `.google_authenticator` also contains 5 one-time **scratch codes** the user prints and stores in a safe. Each one works once, then invalidates. This is the recovery path if the TOTP device is lost.

```
sudo tail -5 /home/mfauser/.google_authenticator
```

Step 7 — MFA factor mapping

Factor (Server+ 3.3)	In this lab	Other examples on a server
Something you know	SSH password	PIN, security questions
Something you have	TOTP from phone app	YubiKey (FIDO2), smart card
Something you are	(not used here)	Fingerprint, iris, face

True MFA = factors from ≥ 2 different categories. Password + scratch code is not MFA (both are *something you know*).

Step 8 — Hardening notes

1. Remove ``nullokb`` from the PAM line once every account has a token enrolled — otherwise users without TOTP slip through.
2. Pair with ``AuthenticationMethods publickey,keyboard-interactive`` so even a stolen TOTP secret is not enough — the attacker also needs the SSH private key.
3. Time skew matters: keep ``chrony`/`ntpd`` running on both server and the phone, or codes will fail (Lab 31).

Free online tools

- ****Google Authenticator PAM (open source)**** — <https://github.com/google/google-authenticator-libpam>
- ****FreeOTP (free app)**** — <https://freeotp.github.io/>
- ****oathtool / oath-toolkit**** — <https://www.nongnu.org/oath-toolkit/>
- ****FIDO Alliance specs**** — <https://fidoalliance.org/>

What you learned

- Adding a TOTP second factor to SSH with ``libpam-google-authenticator``.
- The PAM + `sshd_config` wiring that actually enforces two factors.
- How to generate TOTP codes from a CLI for testing.
- Scratch codes for account recovery.
- The factor-category test that distinguishes real MFA from a pair of passwords.